

ReactJS Hands-On Lab

Exercises 9 – 13 : ES6, JSX, Events & Conditional Rendering

Submitted by: Vishal Kumar

Java Full Stack Engineer (FSE) Training Program

VIT Bhopal

Environment: Windows 11, VS Code, Node.js (npx create-react-app)

Exercise 9 — ES6 Features (cricketapp)

This one is basically a tour of the ES6 stuff that shows up constantly in React code — `let/const`, arrow functions, `map/filter`, destructuring and merging arrays with the spread operator. All of it gets tied together in one small cricket-themed app.

Quick concept refresher

let and const vs var

`var` is function-scoped and gets hoisted in a way that causes weird bugs when it is reused inside loops or blocks. `let` and `const` fix that by being block-scoped — a variable declared inside an `if` block or a `for` loop with `let` simply does not leak outside that block. `const` additionally locks the binding itself, so you cannot reassign it later (though if it holds an object or array, the contents can still be mutated).

ES6 classes and inheritance

ES6 gave JavaScript a cleaner syntax for object-oriented style code — a `class` keyword, a `constructor`, and the `extends` keyword for inheritance, where a child class calls `super()` to run the parent constructor first. React class components lean on exactly this: every class component extends `React.Component`.

Arrow functions

Arrow functions (`item => item * 2`) are shorter than the old `function(item){ return item*2 }` syntax, and — more importantly for React — they do not rebind `this`. That is why class components almost always define their event handlers as arrow functions/class fields, so `this` inside them still refers to the component instance.

`map()`, `filter()` and destructuring

- `map()` transforms every element of an array into something else and returns a brand new array — used everywhere in JSX to turn data into a list of elements.
- `filter()` returns a shorter array containing only the elements that pass a condition.
- Destructuring lets you unpack values out of arrays/objects straight into named variables in one line, e.g. `const [a, b] = someArray`.
- The spread operator (`...`) can expand an array in place — handy for merging two arrays into one without a loop.

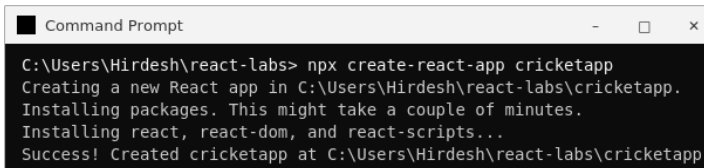
Prerequisites

- Node.js (Windows installer from nodejs.org)
- NPM
- Visual Studio Code

Steps I followed

Step 1: Opened Command Prompt on Windows and scaffolded the app:

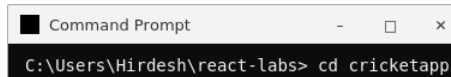
```
npx create-react-app cricketapp
```



```
Command Prompt
C:\Users\Hirdesh\react-labs> npx create-react-app cricketapp
Creating a new React app in C:\Users\Hirdesh\react-labs\cricketapp.
Installing packages. This might take a couple of minutes.
Installing react, react-dom, and react-scripts...
Success! Created cricketapp at C:\Users\Hirdesh\react-labs\cricketapp
```

Step 2: Moved into the project:

```
cd cricketapp
```



```
Command Prompt
C:\Users\Hirdesh\react-labs> cd cricketapp
```

Step 3: Inside src, added a components folder and created ListofPlayers.js — an array of 11 players, rendered with map(), plus a second list filtered with an arrow function for anyone scoring below 70:

```
const players = [
  { name: 'Rohit Sharma', score: 85 },
  { name: 'Virat Kohli', score: 92 },
  { name: 'Shubman Gill', score: 60 },
  { name: 'Shreyas Iyer', score: 55 },
  { name: 'KL Rahul', score: 78 },
  { name: 'Hardik Pandya', score: 65 },
  { name: 'Ravindra Jadeja', score: 40 },
  { name: 'Jasprit Bumrah', score: 15 },
  { name: 'Mohammed Shami', score: 20 },
  { name: 'Kuldeep Yadav', score: 30 },
  { name: 'Ishan Kishan', score: 68 }
];

function ListofPlayers() {
  const playerRows = players.map((player, index) => (
    <li key={index}>{player.name} - {player.score} runs</li>
  ));

  const lowScorers = players.filter(p => p.score < 70);

  return (
    <div className="section">
      <h2>List of Players</h2>
      <ul>{playerRows}</ul>
      <h3>Players who scored below 70</h3>
      <ul>{lowScorers.map((p, i) => <li key={i}>{p.name} - {p.score} runs</li>)}</ul>
    </div>
  );
}
```

Step 4: Created IndianPlayers.js — destructured a flat array of 8 names straight into odd-position and even-position teams, then merged a T20 squad and a Ranji Trophy squad using the spread operator:

```
const allPlayers = ['Rohit Sharma', 'Virat Kohli', 'Shubman Gill', 'Shreyas Iyer',
  'KL Rahul', 'Hardik Pandya', 'Ravindra Jadeja', 'Jasprit Bumrah'];

const [player1, player2, player3, player4, player5, player6, player7, player8] = allPlayers;
```

```
const oddTeamPlayers = [player1, player3, player5, player7];
const evenTeamPlayers = [player2, player4, player6, player8];

const t20Players = ['Yashasvi Jaiswal', 'Suryakumar Yadav', 'Arshdeep Singh'];
const ranjiTrophyPlayers = ['Sarfaraz Khan', 'Tilak Varma', 'Mukesh Kumar'];

// merge feature - spread operator
const mergedSquad = [...t20Players, ...ranjiTrophyPlayers];
```

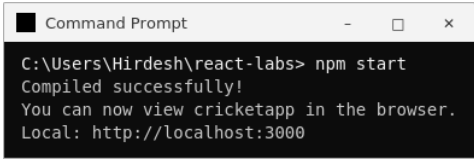
Step 5: In App.js, used a plain if/else on a flag variable to decide which of the two components gets shown on the home page:

```
const flag = true; // flip to false to see IndianPlayers instead

function App() {
  return (
    <div className="App">
      <h1>Cricket App</h1>
      {flag ? <ListofPlayers /> : <IndianPlayers />}
    </div>
  );
}
```

Step 6: Ran the app and checked both states by toggling the flag:

```
npm start
```



```
Command Prompt
C:\Users\Hirdesh\react-labs> npm start
Compiled successfully!
You can now view cricketapp in the browser.
Local: http://localhost:3000
```

Output — when flag = true

Cricket App

List of Players

- Rohit Sharma - 85 runs
- Virat Kohli - 92 runs
- Shubman Gill - 60 runs
- Shreyas Iyer - 55 runs
- KL Rahul - 78 runs
- Hardik Pandya - 65 runs
- Ravindra Jadeja - 40 runs
- Jasprit Bumrah - 15 runs
- Mohammed Shami - 20 runs
- Kuldeep Yadav - 30 runs
- Ishan Kishan - 68 runs

Players who scored below 70

- Shubman Gill - 60 runs
- Shreyas Iyer - 55 runs
- Hardik Pandya - 65 runs
- Ravindra Jadeja - 40 runs
- Jasprit Bumrah - 15 runs
- Mohammed Shami - 20 runs
- Kuldeep Yadav - 30 runs
- Ishan Kishan - 68 runs

Fig 9.1 — `ListofPlayers` rendered via `map()`, low scorers filtered with an arrow function

Output — when flag = false

Cricket App

Indian Players

Odd Team Players

- Rohit Sharma
- Shubman Gill
- KL Rahul
- Ravindra Jadeja

Even Team Players

- Virat Kohli
- Shreyas Iyer
- Hardik Pandya
- Jasprit Bumrah

Merged Squad (T20 + Ranji Trophy)

- Yashasvi Jaiswal
- Suryakumar Yadav
- Arshdeep Singh
- Sarfaraz Khan
- Tilak Varma
- Mukesh Kumar

Fig 9.2 — `IndianPlayers`: odd/even teams via destructuring, merged squad via spread operator

Exercise 10 — JSX (officespacerentalapp)

Small office-rental listing page, used here mainly to practice plain JSX mechanics — creating elements, plugging JS values/objects into markup, looping over a list, and applying inline CSS conditionally based on a value.

A bit of background

What JSX actually is

JSX looks like HTML sitting inside JavaScript, but it is not HTML — it is syntactic sugar that gets compiled (by Babel, as part of ECMAScript tooling) into nested `React.createElement()` calls. `React.createElement('h1', null, 'hello')` and `<h1>hello</h1>` compile down to the exact same thing; JSX just saves you from writing the verbose version by hand.

Rendering JSX to the DOM

Once you have a JSX tree, `createRoot(document.getElementById('root')).render(<App />)` is what actually mounts it into the real page — React takes the element tree, builds the corresponding DOM nodes, and inserts them into that root div in `index.html`.

Expressions and inline CSS inside JSX

Anything inside a single pair of curly braces `{}` in JSX is evaluated as a normal JavaScript expression — a variable, a function call, a ternary, whatever. Inline styles follow the same idea but take a JS object instead of a CSS string, so `style={{ color: 'red' }}` (double braces: outer for the JS expression, inner for the object literal).

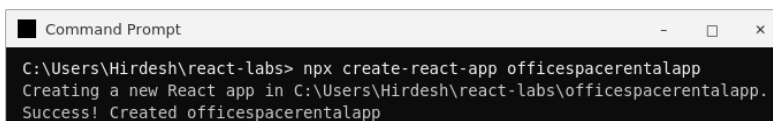
Prerequisites

- Node.js (Windows installer from nodejs.org)
- NPM
- Visual Studio Code

Steps I followed

Step 1: Scaffolded the project from Command Prompt:

```
npx create-react-app officespacerentalapp
```



```
Command Prompt
C:\Users\Hirdesh\react-labs> npx create-react-app officespacerentalapp
Creating a new React app in C:\Users\Hirdesh\react-labs\officespacerentalapp.
Success! Created officespacerentalapp
```

Step 2: Created the page heading using `React.createElement()` directly, just to see the non-JSX form side by side with everything else that uses JSX shorthand:

```
const heading = React.createElement('h1', null, 'Office Space Rental');
```

Step 3: Added an `<img>` tag as the "attribute" example (`src`, `alt`, `width` are all JSX attributes):

```
<img src={officeImage} alt="office space" width="320" />
```

Step 4: Declared an office object and rendered its fields with JSX expressions:

```
const office = {  
  name: 'Skyline Business Hub',  
  rent: 55000,  
  address: 'MP Nagar, Bhopal'  
};
```

Step 5: Declared a list of office objects and looped over it with `map()` to render more rows:

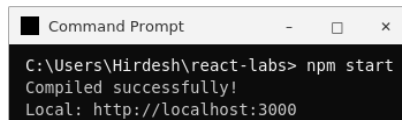
```
const officeList = [  
  { name: 'Skyline Business Hub', rent: 55000, address: 'MP Nagar, Bhopal' },  
  { name: 'Orbit Corporate Park', rent: 72000, address: 'Arera Colony, Bhopal' },  
  { name: 'Greenfield Towers', rent: 48000, address: 'Kolar Road, Bhopal' },  
  { name: 'Metro Business Center', rent: 65500, address: 'Hoshangabad Road, Bhopal' }  
];
```

Step 6: Applied conditional inline CSS on the rent value — red under Rs. 60,000, green at or above it:

```
<span style={{ color: office.rent < 60000 ? 'red' : 'green' }}>  
  Rs. {office.rent}/month  
</span>
```

Step 7: Ran the dev server:

```
npm start
```



```
Command Prompt  
C:\Users\Hirdesh\react-labs> npm start  
Compiled successfully!  
Local: http://localhost:3000
```

Note: The office photo in the screenshot is a simple inline SVG placeholder since this write-up was put together without live internet access. In the actual project this would just be a real photo URL or a local asset import — the `img` tag and the conditional-color logic work exactly the same either way.

Output

Office Space Rental



Featured Office

Name: Skyline Business Hub

Address: MP Nagar, Bhopal

Rent: Rs. 55000/month

All Available Offices

- Skyline Business Hub — MP Nagar, Bhopal — **Rs. 55000**
- Orbit Corporate Park — Arera Colony, Bhopal — **Rs. 72000**
- Greenfield Towers — Kolar Road, Bhopal — **Rs. 48000**
- Metro Business Center — Hoshangabad Road, Bhopal — **Rs. 65500**

Fig 10.1 — Rent shown in red for offices under Rs. 60,000, green otherwise

Exercise 11 — Event Handling (eventexamplesapp)

This lab is all about wiring up click and form events in React — a counter, a button that calls a function with an argument, a synthetic-event demo, and a small currency convertor that reacts to a form submit.

Concepts covered

React events vs plain DOM events

React events are named in camelCase (onClick, onChange, onSubmit) instead of lowercase (onclick), and you pass an actual function reference rather than a string. Apart from that, they behave a lot like the events you already know from plain JavaScript.

SyntheticEvent

Every event handler in React does not receive the raw browser event directly — it receives a SyntheticEvent, which is a cross-browser wrapper React builds around the native event. It has the same interface as a normal DOM event (preventDefault(), target, etc.) but behaves consistently no matter which browser is running the app, which used to matter a lot more before browsers converged on standards.

The this keyword in event handlers

Inside a class component, a method written with the class-field arrow syntax (handleClick = () => {...}) automatically keeps this pointing at the component instance, so this.setState() inside a handler works without needing to manually .bind(this) in the constructor — that used to be a common source of "cannot read property of undefined" bugs before this syntax was standard.

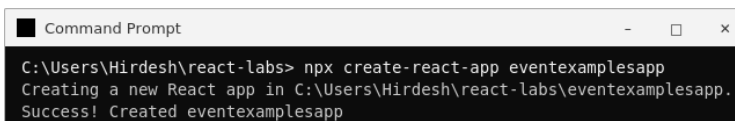
Prerequisites

- Node.js (Windows installer from nodejs.org)
- NPM
- Visual Studio Code

Steps I followed

Step 1: Scaffolded the project:

```
npx create-react-app eventexamplesapp
```



```
Command Prompt
C:\Users\Hirdesh\react-labs> npx create-react-app eventexamplesapp
Creating a new React app in C:\Users\Hirdesh\react-labs\eventexamplesapp.
Success! Created eventexamplesapp
```

Step 2: Built CounterButtons.js — Increment calls two methods (bump the counter, then show a greeting), Decrement just lowers the count:

```
class CounterButtons extends Component {
  constructor(props) {
```

```

    super(props);
    this.state = { count: 0, greeting: '' };
  }

  incrementCount = () => this.setState(prev => ({ count: prev.count + 1 }));
  sayHello = () => this.setState({ greeting: 'Hello, hope you are having a great day!' });

  handleIncrementClick = () => {
    this.incrementCount();
    this.sayHello();
  };

  handleDecrementClick = () => this.setState(prev => ({ count: prev.count - 1 }));

  render() {
    return (
      <div className="section">
        <h2>Counter</h2>
        <p>Count: <b>{this.state.count}</b></p>
        <button onClick={this.handleIncrementClick}>Increment</button>
        <button onClick={this.handleDecrementClick}>Decrement</button>
        {this.state.greeting && <p className="greeting">{this.state.greeting}</p>}
      </div>
    );
  }
}

```

Step 3: Built WelcomeButton.js — the click handler calls a function that takes "welcome" as its argument:

```

sayWelcome = (text) => {
  this.setState({ message: `${text.charAt(0).toUpperCase() + text.slice(1)}! Glad to have you here.` });
};

// in render():
<button onClick={() => this.sayWelcome('welcome')}>Say Welcome</button>

```

Step 4: Built SyntheticEventDemo.js — clicking the button reads the SyntheticEvent, calls preventDefault() on it, and shows a message:

```

handlePress = (e) => {
  e.preventDefault();
  this.setState({ clicked: true });
};

// in render():
<button onClick={this.handlePress}>Press Me</button>
{this.state.clicked && <p className="greeting">I was clicked</p>}

```

Step 5: Built CurrencyConvertor.js — a small form; submitting it converts the entered rupee amount to euros:

```

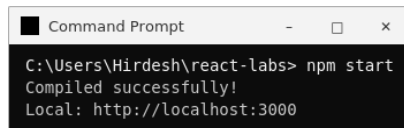
handleSubmit = (e) => {
  e.preventDefault();
  const rate = 0.011; // approx INR -> EUR
  const euroValue = (parseFloat(this.state.rupees) * rate).toFixed(2);
  this.setState({ euro: isNaN(euroValue) ? 0 : euroValue });
}

```

```
};  
  
<form onSubmit={this.handleSubmit}>  
  <input type="number" value={this.state.rupees} onChange={this.handleChange} />  
  <button type="submit">Convert</button>  
</form>
```

Step 6: Dropped all four components into App.js and ran it:

```
npm start
```



```
Command Prompt  
C:\Users\Hirdesh\react-labs> npm start  
Compiled successfully!  
Local: http://localhost:3000
```

Output

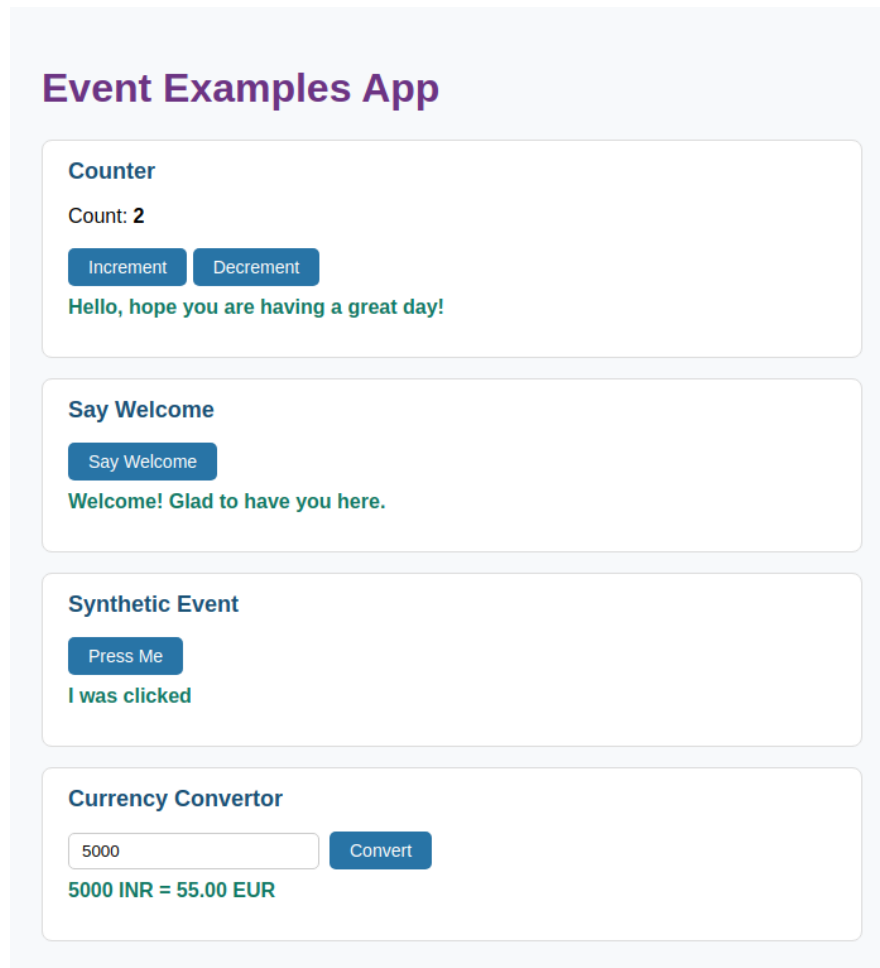


Fig 11.1 — Counter after two Increment clicks, welcome message, synthetic-event click, and a converted amount

Exercise 12 — Conditional Rendering (ticketbookingapp)

A flight-booking style page where the Guest view (browse flights) and the logged-in User view (book a ticket) swap depending on login state — a pretty common real-world pattern.

Concepts covered

Conditional rendering, the basic idea

React does not have a special "if" tag — you just use regular JavaScript conditionals (if/else, ternaries) inside a component to decide which JSX to return or which chunk of markup to include. Since JSX expressions are just values, an if statement can return one element tree or another depending on a piece of state.

Element variables

Instead of duplicating a big chunk of JSX inside both branches of an if, it is common to build up a variable that holds "whichever component should show right now" and then just drop that variable into the return statement once — keeps the render method a lot less repetitive.

Preventing a component from rendering at all

Returning null from a component's render (or from a function component) tells React to render nothing for that spot — useful for things like a warning banner that should only exist under specific conditions.

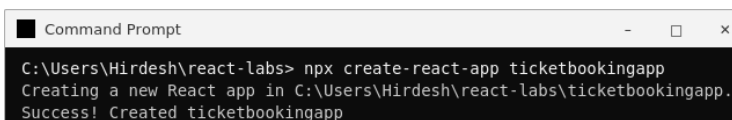
Prerequisites

- Node.js (Windows installer from nodejs.org)
- NPM
- Visual Studio Code

Steps I followed

Step 1: Scaffolded the project:

```
npx create-react-app ticketbookingapp
```



```
Command Prompt
C:\Users\Hirdesh\react-labs> npx create-react-app ticketbookingapp
Creating a new React app in C:\Users\Hirdesh\react-labs\ticketbookingapp.
Success! Created ticketbookingapp
```

Step 2: Built GuestPage.js — a simple table of flights that anyone can see, logged in or not:

```
const flights = [
  { flightNo: 'AI-202', from: 'Bhopal', to: 'Delhi', time: '07:15 AM', fare: 4200 },
  { flightNo: 'SG-441', from: 'Bhopal', to: 'Mumbai', time: '11:40 AM', fare: 5100 },
  { flightNo: '6E-118', from: 'Bhopal', to: 'Bengaluru', time: '05:20 PM', fare: 6300 }
];

function GuestPage() {
  return (
```

```

    <div className="section">
      <h2>Available Flights</h2>
      <p className="hint">You are browsing as a guest. Log in to book a ticket.</p>
      <table>...</table>
    </div>
  );
}

```

Step 3: Built `UserPage.js` — only shown once logged in, with a Book Ticket button:

```

function UserPage() {
  return (
    <div className="section">
      <h2>Welcome back!</h2>
      <p className="hint">You are logged in. Select a flight below and confirm your booking.</p>
      <button className="book-btn">Book Ticket - AI 202 (Bhopal to Delhi)</button>
    </div>
  );
}

```

Step 4: In `App.js`, kept an `isLoggedIn` flag in state and toggled it from the Login/Logout button, deciding which page to render with a ternary:

```

class App extends Component {
  constructor(props) {
    super(props);
    this.state = { isLoggedIn: false };
  }

  handleLogin = () => this.setState({ isLoggedIn: true });
  handleLogout = () => this.setState({ isLoggedIn: false });

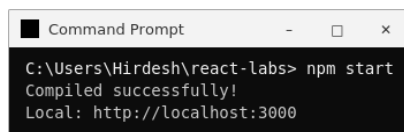
  render() {
    const { isLoggedIn } = this.state;
    return (
      <div className="App">
        <h1>Ticket Booking App</h1>
        {isLoggedIn
          ? <button onClick={this.handleLogout}>Logout</button>
          : <button onClick={this.handleLogin}>Login</button>}

        {isLoggedIn ? <UserPage /> : <GuestPage />}
      </div>
    );
  }
}

```

Step 5: Ran the app and tested both the guest and logged-in states by clicking Login/Logout:

```
npm start
```



```

Command Prompt
C:\Users\Hirdesh\react-labs> npm start
Compiled successfully!
Local: http://localhost:3000

```

Output — Guest view (not logged in)

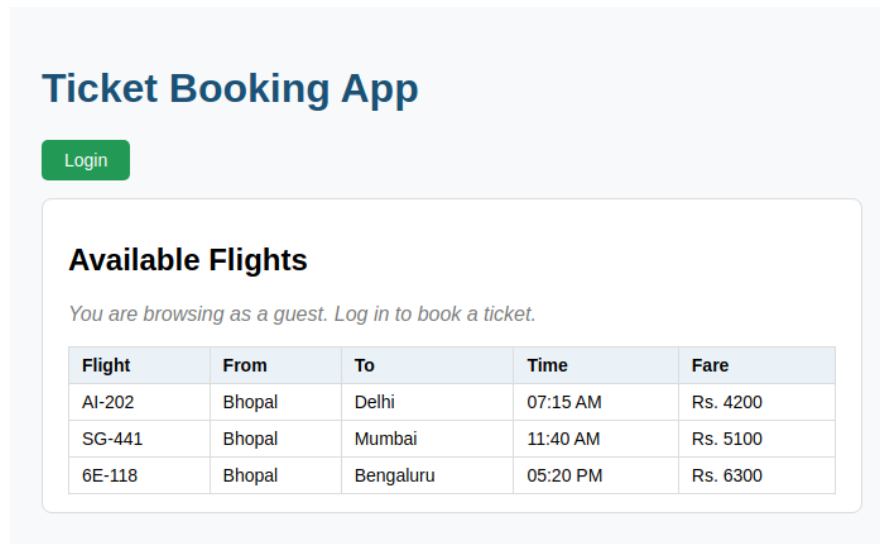


Fig 12.1 — Guest sees the flight table and a Login button

Output — User view (after clicking Login)

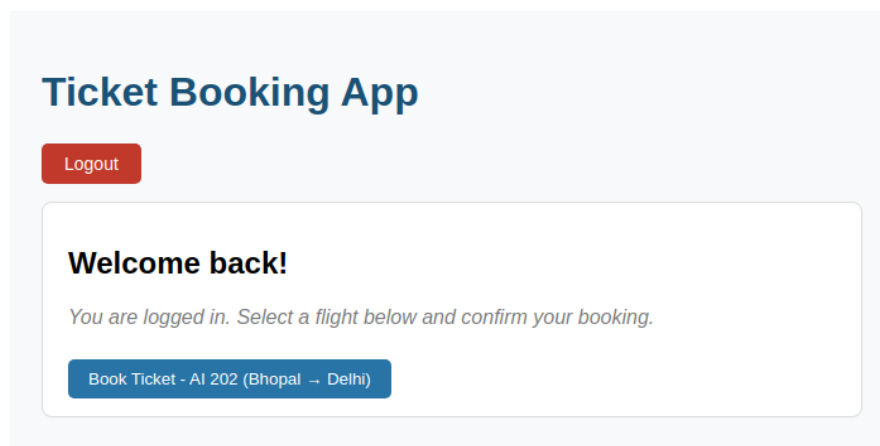


Fig 12.2 — Logged-in view swaps to the booking page and a Logout button

Exercise 13 — More Conditional Rendering, Lists & Keys (bloggerapp)

Same underlying idea as the previous exercise, but this time the point was to actually use several different ways of writing a condition in JSX rather than sticking to just one, across three small components — Book Details, Blog Details and Course Details.

Concepts covered

The different flavours of conditional rendering

- if / else before the return — used in BookDetails, where an early return exits the function with one JSX tree or another depending on stock availability.
- Element variables — used in BlogDetails, where a content variable is assigned different JSX depending on a condition, then rendered once at the bottom.
- The ternary operator (cond ? a : b) — used in CourseDetails for the enrollment status line.
- The && short-circuit trick — also in CourseDetails; {seatsLeft > 0 && <p>...</p>} only renders the paragraph when the left-hand side is truthy, and renders nothing (not even a stray "false" on screen) otherwise.

Rendering lists and why keys matter

Whenever you map() over an array to produce a list of elements, each element needs a unique key prop. React uses these keys behind the scenes to figure out which items were added, removed, or reordered between renders, instead of re-creating the whole list from scratch every time — using the array index as a fallback key is fine for a static list like these, but for a list that can be reordered a stable ID is safer.

Extracting components

Book Details, Blog Details and Course Details are each pulled out into their own file rather than living inline in App.js — keeps each one testable and readable on its own, and is exactly the kind of split you would keep doing as a real app grows.

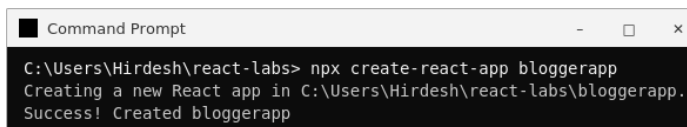
Prerequisites

- Node.js (Windows installer from nodejs.org)
- NPM
- Visual Studio Code

Steps I followed

Step 1: Scaffolded the project:

```
npx create-react-app bloggerapp
```



```
Command Prompt
C:\Users\Hirdesh\react-labs> npx create-react-app bloggerapp
Creating a new React app in C:\Users\Hirdesh\react-labs\bloggerapp.
Success! Created bloggerapp
```


Step 2: BookDetails.js — if / else before the return:

```
function BookDetails(props) {
  const { available } = props;

  if (!available) {
    return (
      <div className="card">
        <h3>Book Details</h3>
        <p>Currently out of stock.</p>
      </div>
    );
  }

  return (
    <div className="card">
      <h3>Book Details</h3>
      <p>"Atomic Habits" by James Clear - available now.</p>
    </div>
  );
}
```

Step 3: BlogDetails.js — element variable:

```
function BlogDetails(props) {
  const { hasPosts } = props;
  let content;

  if (hasPosts) {
    content = <p>Latest post: "10 tips to crack the TCS NQT verbal section."</p>;
  } else {
    content = <p>No blog posts published yet.</p>;
  }

  return (
    <div className="card">
      <h3>Blog Details</h3>
      {content}
    </div>
  );
}
```

Step 4: CourseDetails.js — ternary plus the && short-circuit pattern:

```
function CourseDetails(props) {
  const { enrolled, seatsLeft } = props;
  return (
    <div className="card">
      <h3>Course Details</h3>
      <p>Status: {enrolled ? 'Enrolled' : 'Not Enrolled'}</p>
      {seatsLeft > 0 && <p>Seats remaining: {seatsLeft}</p>}
      {seatsLeft === 0 && <p>This batch is full.</p>}
    </div>
  );
}
```

Step 5: Rendered all three in App.js and ran the dev server:

```
npm start
```

```
Command Prompt
C:\Users\Hirdesh\react-labs> npm start
Compiled successfully!
Local: http://localhost:3000
```

Output

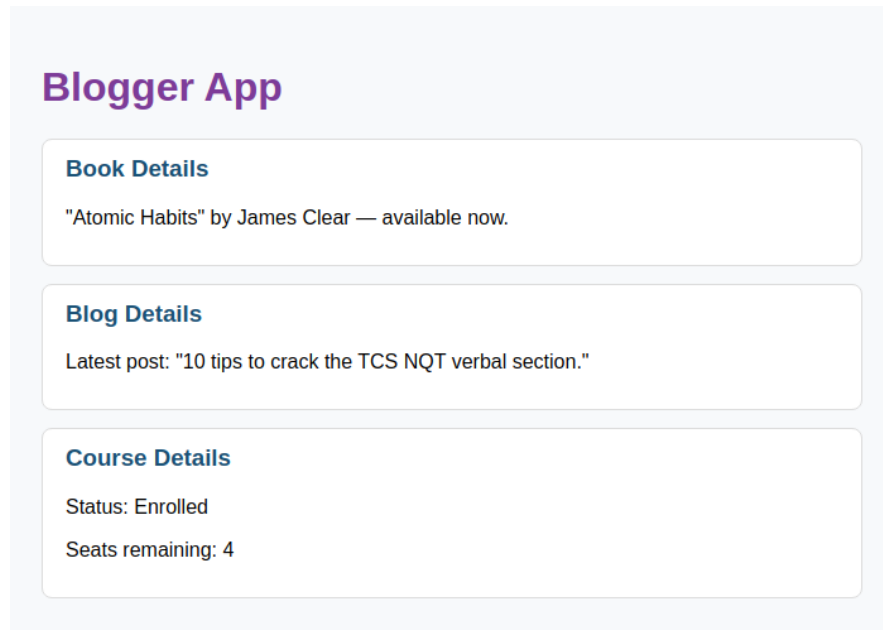


Fig 13.1 — Book Details, Blog Details and Course Details, each using a different conditional-rendering style